

Name: _____

Write your name and computing ID above. Write your computing ID at the top of each page in case pages get separated. Sign the honor pledge below.

Generally, we will not answer questions about the exam during the exam time. If you think a question is unclear and requires additional information to answer, please explain how in your answer. For multiple choice questions, write a * next to the relevant option(s) along with your explanation.

On my honor as a student I have neither given nor received aid on this exam.

1. (6 points) The Makefile shown correctly builds the `pool` program when running `make pool`. Re-write the Makefile in the provided space to still correctly build `pool` using the same `make` command while not using any of the strings: “`pool.c`”, “`queue.c`”, “`pool.o`”, or “`queue.o`”.

Existing Makefile:

```
pool.o: pool.c pool.h
    gcc -o pool.o -c pool.c

queue.o: queue.c pool.h
    gcc -o queue.o -c queue.c

pool: pool.o queue.o
    gcc pool.o queue.o -o pool
```

Blank Makefile to fill in: (You may not need all lines.)

```
OBJS = pool.o queue.o
```

```
-----: -----
    -----
    -----

-----: -----
    -----
    -----

-----: -----
    -----
    -----
```

2. Consider the following incomplete code to run two programs, `./foo` and `./bar` simultaneously. Each program should separately read the contents of `in.txt` as input, and have their output interleaved in `out.txt`. The _____s marked with a comment 'BLANK (letter)' represent places where code may be omitted and asked about below. (It is possible not all blanks need to have code added for the function to work.)

```
int fd1 = open("in.txt", O_RDONLY);
int fd2 = open("out.txt", O_WRONLY);
int fd3 = open(_____); /* BLANK A */
pid_t p = fork();
if (p == 0) {
    char argv[] = {"./foo", NULL};
    _____ /* BLANK B */
    close(fd1); close(fd2); close(fd3);
    execv("./foo", argv);
    _____ /* BLANK C */
}
_____ /* BLANK D */
pid_t q = fork();
if (q == 0) {
    char argv[] = {"./bar", NULL};
    _____ /* BLANK E */
    close(fd1); close(fd2); close(fd3);
    execv("./bar", argv);
    _____ /* BLANK F */
}
_____ /* BLANK G */
```

- (a) (3 points) Where would be most appropriate to place calls to `waitpid()`?
- ☐ in BLANK C for p and in BLANK F for q
 - ☐ in BLANK D for p and in BLANK G for q
 - ☐ in BLANK F for both p and q
 - ☐ in BLANK G for both p and q
- (b) (3 points) What would be most appropriate to place in BLANK A?
- ☐ `"out.txt", O_WRONLY`
 - ☐ `"in.txt", O_RDONLY`
 - ☐ none of these; a third open call is not needed
- (c) (4 points) What would be most appropriate to place in BLANK B?
- ☐ `dup2(STDIN_FILENO, fd1); dup2(STDOUT_FILENO, fd2);`
 - ☐ `dup2(fd1, STDIN_FILENO); dup2(fd2, STDOUT_FILENO);`
 - ☐ `STDIN_FILENO = dup(fd1); STDOUT_FILENO = dup(fd2);`
 - ☐ `dup2(fd1, STDIN_FILENO); dup2(fd3, STDOUT_FILENO);`
 - ☐ `dup2(STDIN_FILENO, STDOUT_FILENO); dup2(fd2, fd1);`
- (d) (3 points) What would be most appropriate to place in BLANK C?
- ☐ `close(STDIN_FILENO); close(STDOUT_FILENO);`
 - ☐ `fork();`
 - ☐ `_Exit(1);`
 - ☐ `return;`
 - ☐ `p = -1;`

3. Suppose on a single-core system we run an interactive program which does the following:
- at time 0 s, the key ‘enter’ is pressed and the program immediately displays something and waits for another keypress
 - at time 0.1 s, the key ‘y’ is pressed on the keyboard and the program immediately queries some data structures to figure out what to do, and then starts opening a file, but must wait for the OS to read its permission from disk
 - at time 0.25 s, the OS confirms program has permission to open the file, then, after the program opens the file, it immediately starts reading it, but must wait for the OS to read that data from the disk
 - at time 0.6 s, the OS reads data from disk and sends it to the program, then the program spends 0.1 s computing something based on it
 - at time 0.7 s, the program finishes its computation and displays the result to the screen

And alongside that program we run a program similar to the one in emptyloop represented by the following pseudocode:

```
previous_time = RecordTime();
while (true) {
    current_time = RecordTime();
    Print(current_time - previous_time);
    previous_time = current_time;
}
```

- (a) Suppose between time -0.001 s and time 0.11 s, the emptyloop-style program outputs:
- 100000 measurements between 30 and 100 ns
 - 106 measurements between 200 and 1000 ns
 - 2 measurements over 100000 ns
- i. (2 points) A timer interrupt is most likely to appear in _____.
☐ one of the 100000 measurements between 30 and 100 ns
☐ one of the 106 measurements between 200 and 1000 ns
☐ one of the 2 measurements over 100000 ns
- ii. (2 points) A interrupt triggered by a keyboard is most likely to appear in _____.
☐ one of the 100000 measurements between 30 and 100 ns
☐ one of the 106 measurements between 200 and 1000 ns
☐ one of the 2 measurements over 100000 ns
- (b) (3 points) The emptyloop-style program almost always outputs very short time measurements. During which of the following time periods, are we most likely to see it output its longest measurements?
- ☐ between times 0 s and 0.1 s
 - ☐ between times 0.1 s and 0.25 s
 - ☐ between times 0.25 s and 0.6 s
 - ☐ between times 0.6 s and 0.7 s
- (c) (3 points) Assume we didn’t have the emptyloop-style program and only ran the interactive program. At which times is the interactive program context switched to?

4. (4 points) Suppose a system has:

- 4096-byte pages
- 48-bit virtual addresses
- 36-bit physical addresses
- 8-byte page table entries
- a 4-level page table structure with 512 entries in page tables at each level
- a 16-entry, 2-way TLB
- a 1024-set, 64-byte-block, direct-mapped data cache

What is the size in bits of a single TLB tag?

5. Suppose machine A and machine B send messages to each other using a reliability protocol similar to what we discussed in lecture:

- each message is divided into 100 byte chunks;
- each 100 byte chunk is sent in a data packet containing:
 - a sequence number (that is one more than the last sequence number used by the sender)
 - the 100 byte chunk; and
 - a checksum
- for each data packet received, the receiver sends back an acknowledgment packet containing
 - the sequence number that was just received (even if not all previous sequence numbers have been received); and
 - a checksum
- if no acknowledgment packet for a particular data packet's sequence number is received 50 ms after a data packet is sent, the data packet is resent

Unlike the basic scheme discussed in lecture, when a message is to be sent, all the data packets are sent as fast as possible. In the network, it takes 20 ms for B to receive a packet sent by A or vice-versa, and A or B can only transmit a packet 1 ms after it transmits its previous packet. (So, for example, A can send 2 packets to B, the first of which would be received in 20 ms, the second of which would be received in 21 ms.)

Suppose A sends a 600 byte message to B. Once B receives the whole message, it sends back a 400 byte reply message to A.

- (a) (4 points) How long (from when A starts sending data) until A receives the reply message if there are no lost or corrupted packets.

- (b) (4 points) Suppose every 3rd packet sent by A starting with the second is corrupted. How long will it be until A receives the reply message?

- (c) (4 points) Suppose we modified the protocol to add encryption and authentication: instead of sending each 100 byte chunk, we'd send the chunk encrypted with the public key of the recipient. What would still be possible for a machine-in-the-middle attacker to do in this scenario? **Select all that apply.**

- ☐ change the contents of the message B receives from A to something of the attacker's choice
- ☐ change the contents of half of A's message without changing the rest, without knowing the contents of A's original message
- ☐ if the attacker knows the format of A's message, determine the value of a particular number in that message
- ☐ make it appear (based on acknowledgments) to A that B received a message that B did not

6. Suppose we construct a 7-stage pipelined processor with the following stages:

- fetch part 1
- fetch part 2
- decode (register read)
- execute
- memory part 1
- memory part 2
- writeback

In this processor, the stages do the same work as the five-stage pipelined processor we discussed in lecture, except that the fetch and memory operations are split into two stages.

Assume the processor predicts all conditional jumps as *not taken* and uses forwarding (combined, when necessary with stalling) to resolve data hazards.

(a) (6 points) Consider the following assembly snippet:

```
movq (%r8), %r9
addq %r9, %r10
addq %r9, %r11
movq %r10, 8(%r11)
```

If the first `movq` instruction starts its fetch part 1 stage in cycle 0, then the last `movq` will writeback in cycle:

(b) (4 points) Consider the following assembly snippet:

```
movq (%r8), %r10
xorq %r10, %r11
cmpq %r9, %r8
jne foo
addq %r9, %r10
subq %r8, %r10
```

Assume the `jne` is not taken (and predicted correctly). When the above assembly snippet runs, we would expect which of the following to occur? **Select all that apply.**

- ☐ %r8's value is forwarded from `movq` to `cmpq`
- ☐ %r8's value is forwarded from `cmpq` to `subq`
- ☐ %r9's value is forwarded from `addq` to `subq`
- ☐ %r10's value is forwarded from `movq` to `xorq`
- ☐ %r10's value is forwarded from `movq` to `addq`
- ☐ %r10's value is forwarded from `movq` to `subq`
- ☐ %r10's value is forwarded from `addq` to `subq`

7. Consider an out-of-order processor with the following execution units:

- two pipelined integer ALUs that take two cycles to compute the result of an addition
- one pipelined integer ALU that takes three cycles to compute the result of a multiplication

The processor can use the result of an ALU operation to start another one the cycle after it is computed (so $a + b + c$ can be computed in four cycles).

(a) (2 points) What is the minimum time, in cycles, needed to compute

$$d + e + f + g$$

on this processor?

(b) (2 points) What is the minimum time, in cycles, needed to compute

$$((a \times (b + c)) + (d \times (e + f))) + h$$

on this processor?

(c) (5 points) Consider the following assembly snippet:

1. `addq %r8, %r9`
2. `movq (%r8), %r10`
3. `subq %r9, %r10`
4. `movq (%r10), %r11`
5. `movq %r10, (%r9)`

After register renaming, _____. **Select all that apply.**

- ☐ one of instruction 1's inputs will be the same as one of instruction 4's inputs
- ☐ one of instruction 2's outputs will be the same as one of instruction 4's inputs
- ☐ one of instruction 3's outputs will be the same as one of instruction 5's inputs
- ☐ one of instruction 5's inputs will be the same as one of instruction 3's inputs
- ☐ one of instruction 5's inputs will be the same as one of instruction 3's outputs

8. A system uses a two-level page table with 128 byte pages. Each page table entry (PTE) is 8 bytes and each page table has 16 PTEs. The system uses a single level 32 byte direct-mapped cache with 8 byte blocks. The system has no TLB, so all virtual memory accesses involve accessing page table entries stored in physical memory, then, based on those page table entries, accessing the corresponding physical memory location for data.

Level 1 page table located at physical address 0x1c00 (first five rows):

valid	read	write	physical page number
0			
1	1	1	0x39
1	1	1	0x72
0			
0			
...			

Cache:

valid	tag	data
1	0x276	0x11 0x22 0x33 0x44 0x55 0x66 0x77 0x88
0	0x0F0	0x00 0x00 0x88 0x88 0x00 0x00 0x88 0x88
0	0x801	0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
1	0x1C9	0x51 0x52 0x53 0x54 0x55 0x56 0x57 0x58

For the question below, it may help to know what the physical page number and tag values above are when they are multiplied by powers of two as shown in the following table.

value	$\times 8 = 2^3$	$\times 16 = 2^4$	$\times 32 = 2^5$	$\times 128 = 2^7$
0x39	0x1c8	0x390	0x720	0x1c80
0x72	0x390	0x720	0xe40	0x3900
0xf0	0x780	0xf00	0x1e00	0x7800
0x1c9	0xe48	0x1c90	0x3920	0xe480
0x276	0x13b0	0x2760	0x4ec0	0x13b00
0x801	0x4008	0x8010	0x10020	0x40080

It may also help to know that multiplying by 2^x is the same as left-shifting by x .

- (a) (10 points) What is a **virtual address** that will definitely result in a cache hit given the cache contents and page table contents above? You may leave your answer as a unsimplified arithmetic expression.

9. Consider the following C snippet:

```
1  long current_user;
2
3  struct ServerSocket {
4      long permitted_user;
5      long is_open;
6      long pending_connection_id;
7  };
8
9  struct ServerSocket server_sockets[65536];
10
11 struct PendingConnection {
12     long from_address;
13     long fd;
14 };
15
16 struct PendingConnection pending_connections[10000];
17
18 bool IsAccessible(long port) {
19     if (port < 65536 && port >= 0 &&
20         server_sockets[port].is_open == 1 &&
21         server_sockets[port].permitted_user == current_user) {
22         return true;
23     } else {
24         return false;
25     }
26 };
27
28 long GetPendingConnectionAddress(long port) {
29     if (IsAccessible(port)) {
30         long id = server_sockets[port].pending_connection_id;
31         if (id == -1)
32             return -1;
33         else
34             return pending_connections[id].from_address;
35     } else {
36         return -1;
37     }
38 }
39
40
```

For the following questions, assume that:

- the system this runs out does not use virtual memory and has a 512 KiB (2^{19}), 4-way data cache with 256-byte (2^8 byte) blocks
- the array `server_sockets` is located at address 0x400100
- the array `pending_connections` is located at address 0x580200
- a `struct ServerSocket` has a size of 24 bytes
- a `struct PendingConnection` has a size of 16 bytes

- (a) (2 points) How many set index bits does the data cache have? You may leave your answer as an unsimplified arithmetic expression.

- (b) (4 points) When calling `GetPendingConnectionAddress(X)`, an access to `server_sockets[port].is_open` evicts from cache set 202. What is a possible value of `X`? You may leave your answer as an unsimplified arithmetic expression.

- (c) `GetPendingConnectionAddress()` is vulnerable to a Spectre-style vulnerability where an attacker who can control the value of `port` can learn information because of what `pending_connection[id].from_address` evicts from the cache.

Suppose the attacker wants to use this vulnerability to learn about the contents of memory address `0x600010`.

- i. (2 points) Before the attacker runs the `GetPendingConnectionAddress()` function to measure the cache eviction, which of the following preparation steps would be useful for the attacker to make?

Select all that apply.

- ☐ arranging for `server_socket` elements to have a particular value for `pending_connection_id`
- ☐ arranging for `IsAccessible(port)` to be true for the port number they'll use in the attack mod 65536
- ☐ calling `GetPendingConnectionAddress()` repeatedly with arguments for which `IsAccessible(port)` is true
- ☐ identifying a port for which `IsAccessible(port)` is false (but port is between 0 and 65536)

- ii. (4 points) The attacker runs the attack in an attempt to learn about an 8-byte value at memory address `0x600010`, supplying an appropriate value of `port` to target that address. The attacker discovers that the access to `from_address` evicts from cache set 257. Based on this, what is a possible value of the 8-byte value at memory address `0x600010`?

10. (12 points) Recall that with *barriers* when a thread waits on a barrier, it waits for all the threads to also wait on the barrier before proceeding. Consider an extension of a barrier where we have variable number of threads in each iteration of the barrier.

A (small) datacenter monitoring service needs to periodically check that machines are running. These checks happen in stages. On odd stages, the frequently checked machines are checked. On even stages all machines are checked. All machines for a stage must be checked before moving to the next stage. Two threads call `CheckFrequently()` and one thread calls `CheckHalfAsOften()`.

Complete the following implementation of this modified barrier using monitors by filling in the blanks:

```
int THREADS_FOR_HALF_AS_OFTEN = 1; int THREADS_FOR_FREQUENT = 2;
pthread_mutex_t lock; pthread_cond_t cv;

long index = 1;
int at_current_index = 0;
void BarrierWaitVariable(long target_index) {
    while (index < target_index) {

        -----
    }
    at_current_index += 1;
    int needed_threads = THREADS_FOR_FREQUENT;
    if (target_index % 2 == 0) needed_threads += THREADS_FOR_HALF_AS_OFTEN;
    if (at_current_index == needed_threads) {
        index += 1;
        at_current_index = 0;

        -----
    } else {

        while (-----) {

            -----
        }
    }
}

void CheckFrequently() {
    // ... Code to check on the machine not shown.
    pthread_mutex_lock(&lock);
    int target_index = index;
    BarrierWaitVariable(target_index);
    pthread_mutex_unlock(&lock);
}

void CheckHalfAsOften() {
    // ... Code to check on the machine not shown.
    pthread_mutex_lock(&lock);
    int target_index = index;
    if (target_index % 2 == 1) target_index += 1;
    BarrierWaitVariable(target_index);
    pthread_mutex_unlock(&lock);
}
```